

AI Memory Stack

Installation Checklist — from blank machine to a running local AI · v3.0

How to use this: print it, keep it next to the machine, tick every box in order. Fill in the blanks as you go — usernames, hostnames, wifi — so everything is in one place. Never skip ahead: each block builds on the previous one.

Passwords: a password manager (Bitwarden, 1Password, KeePassXC) is the first choice. The lines below say "password or hint — your choice". Be aware: filled in, this sheet is a complete access kit to the machine (passwords + IP + username on one page). Lock it away. Shred it on disposal. Never photograph it.

Step 0 — Pick a machine (5 min)

Windows is supported via WSL2 only (the script prints exact instructions if you run it there). For a clean local-first install, macOS or Linux Mint is simpler.

RAM	What you can run
8 GB (minimum)	3B models — works, but limited quality
16 GB	7–14B models — good daily assistant
32–48 GB (recommended)	32–35B models — strong local agent

Also needed: 60+ GB free disk (20 GB absolute minimum), internet for the install.

■ I am installing on: ■ Mac ■ PC with Linux Mint

Machine name / location: _____

Tip: start with ONE machine. The second install takes 15 minutes with this same list.

Track A — Mac: clean reinstall (45–60 min)

Skip to Track B if you chose Linux Mint.

A1. Enter Recovery Mode — depends on your Mac's chip

- Back up anything you want to keep (external drive or another machine)
- Find your chip type: → About This Mac. Tick one: ■ Apple Silicon (M1–M4) ■ Intel
- **Apple Silicon:** shut down fully → press and HOLD the power button until 'Loading startup options' → Options → Continue
- **Intel:** shut down → press power, then immediately hold **Cmd+R** until the Apple logo appears

A2. Erase and reinstall macOS

- Disk Utility → select 'Macintosh HD' → Erase (format: APFS) → quit Disk Utility
- Reinstall macOS → follow the wizard (20–40 min, several restarts)

A3. First-run setup — WITHOUT cloud

- Language / country / keyboard → connect to wifi

Wifi network: _____

- At 'Sign in with your Apple ID': choose **Set Up Later / Skip** — macOS requires NO account
- Create the local computer account:

Full name: _____

Account name (short, lowercase): _____

Password or hint (your choice — see warning, p.1): _____

- Decline: Siri analytics, usage data sharing, Screen Time (all can be enabled later)
- FileVault (disk encryption): recommended if theft is a risk. If enabled, store the recovery key in your password manager
- On the desktop: run Software Update (→ System Settings → General → Software Update)
- **Power settings (do this NOW — sleep kills downloads and remote access):** System Settings → search 'sleep' → prevent sleeping when display is off / set sleep to Never. Screen turning off is fine — system sleep is not
- Open **Terminal** (Cmd+Space, type 'terminal', Enter) → go to **Step 2** on page 4

On Mac the script will pause and tell you exactly which security popup to expect (Xcode tools ~5–8 min, 'ollama was blocked', firewall). That is normal — it waits for you. One password prompt is normal on macOS: the Homebrew installer asks for your account password in the terminal (not a popup) the first time. Your first macOS account is an admin account — keep it that way on this machine; the tools require it. You may also see a notification 'Background items added' (Ollama) — that is expected.

Track B — Linux Mint on a PC (60–90 min)

B1. Create the USB installer (on any working computer)

- Download Linux Mint 22 'Cinnamon' ISO from linuxmint.com/download.php
- Download **balenaEtcher** (etcher.balena.io)
- Insert a USB stick (8 GB+ — **everything on it is erased**)
- Etcher: pick the ISO → pick the stick → Flash (5–10 min)

B2. Boot the PC from the stick

- Insert stick, power on, immediately tap the boot-menu key repeatedly:
Common: **F12** (Dell/Lenovo) · **F9** (HP) · **F8/F11** (Asus/MSI) · **Esc** (older)

My PC's boot key turned out to be: _____

- Pick the USB stick → 'Start Linux Mint'
- If it refuses: enter BIOS (**Del** or **F2**) → disable 'Secure Boot' → retry

B3. Install

- Double-click **Install Linux Mint** on the desktop
- Language / keyboard → tick 'Install multimedia codecs'
- Installation type: **Erase disk and install Mint** (if the PC is dedicated)
- Encryption (LVM + encrypt): recommended if theft is a risk — store the passphrase in your password manager (asked at EVERY boot)
- Create the user:

Your name: _____

Computer name (hostname): _____

Username (lowercase): _____

Password or hint (your choice — see warning, p.1): _____

- Install (15–25 min) → restart → remove the stick when asked

B4. After first login

- Update Manager (shield icon) → install ALL updates → restart
- **Power settings (NOW):** Menu → Power Management → set 'suspend when inactive' to Never. Screen blanking is fine — system suspend is not
- **Old PC?** If the BIOS clock was wrong or settings were forgotten: replace the CMOS battery (CR2032, ~2 €) — a dead one also forgets the power setting below
- **BIOS 'Restore on AC Power Loss':** reboot → enter BIOS (Del/F2) → Power settings → set to **Power On** (makes the PC boot by itself after an outage)
- Open **Terminal** (Ctrl+Alt+T) → go to **Step 2**

On Linux the script asks for your login password ONCE at the start ('sudo') to install system packages. That is the only time. Never start the script itself with sudo.

Step 2 — The AI layer (same for Mac and Linux)

Three scripts, run in order. They handle the classic beginner traps (PATH, sudo, dependencies, interruptions) — your job is: run, read, tick.

2.1 Get the scripts and start — exactly like this

- In the machine's web browser, open the project page (github.com/jordglob/local-ai-memory) → green **Code** button → **Download ZIP**. It lands in **Downloads**
- Double-click the ZIP in Downloads so a folder appears (macOS does this automatically)
- In the terminal, go to that folder: **cd ~/Downloads/local-ai-memory-main** (*cd = change directory; press Tab to auto-complete the name*)
- Sanity check — type: **ls ai-memory-*.sh** You should see the script files listed. If 'No such file or directory': you are in the wrong folder — run the cd line again

The installer copies all scripts to a permanent home (~/.Documents/ai-memory/.tools/) — after step 2.2 you can delete the download and every later command uses that path.

2.2 Install (15–25 min; Intel Mac 25–35; slow internet adds 10–20)

- In the terminal: **bash ai-memory-setup.sh**
- Linux: type your password ONCE when asked (sudo)
- Mac: approve the popups when the script pauses and points at them — including the folder-access prompts ('Terminal would like to access Documents/Downloads'): click **OK/Allow**
- Mac + iCloud Desktop&Documents sync: the script detects it and offers ~/ai-memory instead, so your vault stays OFF the cloud — accept
- Answer the two questions: install Hermes Agent? · start Ollama at login? (Enter = yes)
- Wait for the green box: **'Installation complete — no errors'**
- If RED: read the last line, fix, run the same command again — it resumes where it stopped

Done — date/time: _____

2.3 Configure (5–10 min + optional model download)

- Open a NEW terminal window (Cmd+N / Ctrl+Shift+N)
- Run: **bash ~/.Documents/ai-memory/.tools/ai-memory-configure.sh**
- It analyzes your hardware and suggests the best model — Enter to accept
- If it offers a download: **y** (a 35B model is ~20 GB; 30+ min on slow lines)
- API keys: just press **Enter** twice to skip — everything runs locally anyway

Selected model: _____

2.4 Import your history (5–15 min)

- Order your exports first (each takes minutes to arrive by email):
Claude: claude.ai → Settings → Privacy → Export Data · ChatGPT: Settings → Data Controls → Export
- Put the downloaded ZIP(s) in this machine's Downloads folder
- Run: **bash ~/.Documents/ai-memory/.tools/ai-memory-ingest.sh** — it finds exports and local tool histories itself and asks before importing
- 10 sources supported — see them all with **--list-sources**

Conversations imported: _____

2.5 First run — the test

- Run: **hermes chat** (or: **bash ~/.Documents/ai-memory/.tools/resume.sh**)
- Ask about something from your imported history — verify it can find it
- DONE. File this checklist (locked away) or shred it

Machine identity (the install script prints these values — just copy)

Hostname: _____

Local IP: _____

SSH line (ssh user@ip): _____

WireGuard endpoint (domain or IP:51820): _____

Tailscale IP/name (if used): _____

RustDesk ID (if used): _____

Headless node (optional) — reach this machine without a screen

Run this AFTER Step 2, while you still have a screen and keyboard connected. It is the last time you need them.

- Run on your **MAIN computer first**: `bash ~/Documents/ai-memory/.tools/ai-memory-remote.sh` → answer **1) MAIN**. It creates your SSH key (one per client machine, never copied) and shows the public key — add it to GitHub (Settings → SSH keys)

Main machine — key fingerprint (printed by the script): _____

- Then run the same script **on each node** → answer **2) NODE**
- Part 1–2: enables SSH and installs your public key (easiest: the GitHub username option)
- Part 3: verify key login FROM YOUR OTHER COMPUTER before letting it disable password login
- Part 4 — remote networking: the script analyzes your connection and recommends a path. **WireGuard (fully local) is the first choice**; Tailscale is offered for convenience or behind carrier-NAT (CGNAT)
- If WireGuard: forward **UDP 51820** to the hub machine in your router. Note the endpoint on the identity block
- **Verify from OUTSIDE only** — turn off phone wifi, connect over mobile data, test the tunnel. Do NOT trust 'port checker' sites: WireGuard is silent by design and they wrongly say 'closed'
- Part 5 (optional): RustDesk for the graphical moments (macOS popups)
- Part 6: always-on power profile (no sleep, auto-restart after power loss)
- Mac only: HDMI **dummy plug** (~10 €) in a video port — without it remote graphics is slow and blurry
- Mac only: enable autologin (System Settings → Users & Groups) so services return after a reboot — requires FileVault OFF (tradeoff on the Tips page)
- **MANDATORY pull-the-plug test**: shut down → pull the cord → wait 1 min → plug in. The machine must boot and answer SSH by itself. PCs: fix in BIOS if not. Apple Silicon minis: known to sometimes ignore the setting — if so, plan for a physical button press after outages
- Always keep TWO ways in (SSH + one graphical) — an update can break either one

Tips & Tricks (after everything works)

cmux — run several AI agents side by side (macOS)

A native, open-source terminal built on Ghostty's renderer, designed for agent work: a vertical sidebar shows each workspace's git branch and latest notification, and a panel lights up when an agent needs your input. Free, no telemetry, no account. Install AFTER the base setup works: download from **cmux.io** or GitHub (sst/cmux). Linux alternative: Ghostty (same engine, without the agent features) or plain tmux.

Tools that strengthen the AI workflow (optional, open-source)

All install after the base setup; each is one command. **cmux** (macOS) — run several agents side by side. **llama.cpp** — the engine under many runtimes; GGUF conversion and benchmarking. **litellm** — a local proxy giving one OpenAI-style endpoint across Ollama and cloud fallbacks. **whisper.cpp** — local speech-to-text, so voice notes never leave the machine. **ripgrep** (rg) — instant full-text search across the vault. **jq** — inspect the JSON the importers read. A commercial VPN (Mullvad etc.) can be installed too, but it competes with your own WireGuard tunnel for routing — use one at a time.

Syncthing — sync the vault between machines, no cloud

Free, peer-to-peer, encrypted. Install on both machines (**syncthing.net**), share the vault folder (~/.Documents/ai-memory). Your conversations never touch a third-party server. Do NOT sync ~/.hermes between machines — agent state is per-machine.

Backups — two things matter

1) The vault (~/.Documents/ai-memory) — plain files, any backup tool works; even a periodic copy to a USB drive is fine. 2) ~/.hermes — the agent's memory and config. Time Machine (Mac) or Timeshift+home backup (Mint) covers both. Test a restore once.

Update Advisor — built in

The agent's workspace instructions include a read-only update check: ask it to 'check for updates' and it writes a report with exact upgrade commands to 00-Inbox/UPDATES.md. It never upgrades anything by itself — you decide.

Disk encryption (FileVault / LUKS) — an honest tradeoff

On a laptop that leaves the house: enable it (theft protection). On a headless always-on node: it blocks unattended reboots — the machine stops at a pre-boot password screen no remote tool can reach. Pick based on where the machine lives. If your router ever hands out new IPs, set a DHCP reservation for each node so the identity block stays true. Prefer zero cloud? Plain WireGuard replaces Tailscale at the cost of manual key setup.

macOS folder popups (power-user shortcut)

Each first access to Documents/Downloads triggers an Allow prompt. Granting Terminal **Full Disk Access** once (System Settings → Privacy & Security) removes them all — convenient, but it means anything run in that terminal can read everything. Per-folder approval is the safer default; FDA is for those who know the tradeoff.

If something gets stuck

1) Read the last red line — the scripts always say WHAT failed and what to do. 2) Re-run the same command — everything resumes where it stopped. 3) The log lives at ~/.Documents/ai-memory/.tools/setup.log — show it to an AI assistant and troubleshoot together. 4) Nothing in these steps can break the computer — worst case is restarting from Step 2.